Workspace Management System

REST API Documentation

Version 2.0

Includes USER role + FEATURE layer

1. New Changes

This version introduces two major schema changes:

- USER.role — Global system role (admin/ member). Separate from workspace-level role.
- FEATURE entity — A new layer between Project and Task that works. Tasks now belong to a Feature instead of directly to a Project.

Hierarchy Flow

```
User → Workspace → Project → Feature (Epic) → Task → Comment
```

Standard Response Envelope

Success:

```
{
  "status": "success",
  "data": { ... }
}
```

Error:

```
{
  "status": "error",
  "message": "Descriptive error message"
}
```

2. Base Configuration

Base URL

```
https://api.workspace.com/
```

Authentication

All endpoints (except /auth/register and /auth/login) require a Bearer JWT token:

```
Authorization: Bearer <your_jwt_token>
```

Content-Type

```
Content-Type: application/json
```

3. Authentication

POST	/auth/register

Register a new user. Default role is 'member'. Only admins can create users with role 'admin'.

Request Body

```
{
  "name": "Anil Kumar",
  "email": "anil@example.com",
  "username": "anil",
  "password": "SecurePass@123"
}
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "anil",
    "email": "anil@example.com",
    "username": "anil",
    "role": "member",
    "token": "eyJhbGciOiJIUzI1NiIs..."
  }
}
```

Error Response

```
{
  "status": "error",
  "message": "Email already registered"
}
```

POST	/auth/login

Authenticate a user and receive a JWT token.

Request Body

```
{
  "email": "anil@example.com",
  "password": "SecurePass@123"
}
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "anil",
    "email": "anil@example.com",
    "username": "anil",
    "token": "eyJhbGciOiJIUzI1NiIs..."
  }
}
```

Error Response

```
{
  "status": "error",
  "message": "Invalid email or password"
}
```

POST	`/auth/logout`

Invalidate the current JWT token.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "message": "Logged out successfully"
}
```

GET	`/auth/profile`

Get the currently authenticated user's profile.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "anil",
    "email": "anil@example.com",
    "username": "anil",
    "role": "member",
    "created_at": "2026-04-18T10:30:00Z"
  }
```

```
}
```

4. Users

GET	/users

List all users. Admin only.

Authentication: Bearer Token required

Parameters

Field	Type	Required	Description
role	string	No	Filter by role: admin / manager / member

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "users": [
      { "id": 1, "name": "anil", "email": "anil@example.com", "role": "admin" },
      { "id": 2, "name": "ravi", "email": "ravi@example.com", "role": "member" }
    ],
    "page": 1,
    "total": 2
  }
}
```

GET	/users/:id

Get a user by ID.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 1,
    "name": "anil",
    "email": "anil@example.com",
    "username": "anil",
    "role": "member",
    "created_at": "2026-04-18T10:30:00Z"
  }
}
```

PATCH	/users/:id/role

Update a user's global role. Admin only.

Authentication: Bearer Token required

Request Body

```
{ "role": "manager" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": { "id": 2, "role": "manager" }
}
```

5. Workspaces

POST	/workspaces

Create a new workspace. The creator becomes admin of the workspace.

Authentication: Bearer Token required

Request Body

```
{ "name": "Engineering Team" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 10,
    "name": "Engineering Team",
    "created_by": 1,
    "created_at": "2026-04-18T11:00:00Z"
  }
}
```

GET	/workspaces

List workspaces the user belongs to.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "workspaces": [
      { "id": 10, "name": "Engineering Team", "role": "admin" }
    ]
  }
}
```

GET	/workspaces/:id

Get a workspace by ID.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 10,
    "name": "Engineering Team",
    "created_by": 1,
    "members_count": 5,
    "projects_count": 3
  }
}
```

PUT `/workspaces/:id`

Update a workspace.

Authentication: Bearer Token required

Request Body

```
{ "name": "Engineering & Design" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": { "id": 10, "name": "Engineering & Design" }
}
```

DELETE `/workspaces/:id`

Delete a workspace. Admin only.

Authentication: Bearer Token required

Response (200 OK)

```
{ "status": "success", "message": "Workspace deleted" }
```

POST `/workspaces/:id/members`

Add a member to the workspace.

Authentication: Bearer Token required

Request Body

```
{ "user_id": 2, "role": "member" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 25,
    "user_id": 2,
    "workspace_id": 10,
    "role": "member",
    "joined_at": "2026-04-18T11:30:00Z"
  }
}
```

GET	/workspaces/:id/members

List members of a workspace.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "members": [
      { "user_id": 1, "name": "anil", "role": "admin" },
      { "user_id": 2, "name": "ravi", "role": "member" }
    ]
  }
}
```

DELETE	/workspaces/:id/members/:user_id

Remove a member from the workspace.

Authentication: Bearer Token required

Response (200 OK)

```
{ "status": "success", "message": "Member removed" }
```

6. Projects

POST	/workspaces/:workspace_id/projects

Create a project inside a workspace.

Authentication: Bearer Token required

Request Body

```
{ "name": "Mobile App Q2" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 50,
    "name": "Mobile App Q2",
    "workspace_id": 10,
    "created_by": 1,
    "created_at": "2026-04-18T12:00:00Z"
  }
}
```

GET	/workspaces/:workspace_id/projects

List projects in a workspace.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "projects": [
      { "id": 50, "name": "Mobile App Q2", "features_count": 4 }
    ]
  }
}
```

GET	/projects/:id

Get a project by ID.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 50,
    "name": "Mobile App Q2",
    "workspace_id": 10,
    "created_by": 1,
    "features_count": 4,
    "tasks_count": 32
  }
}
```

PUT `/projects/:id`

Update a project.

Authentication: Bearer Token required

Request Body

```
{ "name": "Mobile App Q2 - 2026" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": { "id": 50, "name": "Mobile App Q2 - 2026" }
}
```

DELETE `/projects/:id`

Delete a project.

Authentication: Bearer Token required

Response (200 OK)

```
{ "status": "success", "message": "Project deleted" }
```

7. Features (Epics)

A Feature works like an Epic in Jira — it groups related Tasks under a Project. Each Task must belong to exactly one Feature.

POST	`/projects/:project_id/features`

Create a new Feature (Epic) under a Project.

Authentication: Bearer Token required

Request Body

```
{
  "name": "User Authentication",
  "description": "Login, signup, password reset, OAuth flows",
  "status": "in_progress"
}
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 100,
    "name": "User Authentication",
    "description": "Login, signup, password reset, OAuth flows",
    "project_id": 50,
    "created_by": 1,
    "status": "in_progress",
    "created_at": "2026-04-18T12:30:00Z"
  }
}
```

GET	`/projects/:project_id/features`

List all Features (Epics) in a Project.

Authentication: Bearer Token required

Parameters

Field	Type	Required	Description
status	string	No	Filter by status: planned / in_progress / done / cancelled

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "features": [
      { "id": 100, "name": "User Authentication", "status": "in_progress", "tasks_count":
8 },
      { "id": 101, "name": "Payment Integration", "status": "planned", "tasks_count": 5 }
    ]
  }
}
```

GET	/features/:id

Get a Feature by ID with summary of its Tasks.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 100,
    "name": "User Authentication",
    "description": "Login, signup, password reset, OAuth flows",
    "project_id": 50,
    "status": "in_progress",
    "created_by": 1,
    "created_at": "2026-04-18T12:30:00Z",
    "tasks_summary": {
      "total": 8,
      "todo": 2,
      "in_progress": 4,
      "done": 2
    }
  }
}
```

PUT	/features/:id

Update a Feature.

Authentication: Bearer Token required

Request Body

```
{
  "name": "User Authentication & SSO",
  "description": "Includes SAML / OIDC flows",
  "status": "in_progress"
}
```

Response (200 OK)

```
{
  "status": "success",
  "data": { "id": 100, "name": "User Authentication & SSO", "status": "in_progress" }
}
```

PATCH	`/features/:id/status`

Update only the status of a Feature.

Authentication: Bearer Token required

Request Body

```
{ "status": "done" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": { "id": 100, "status": "done" }
}
```

DELETE	`/features/:id`

Delete a Feature. Cascades to all Tasks under it.

Authentication: Bearer Token required

Response (200 OK)

```
{ "status": "success", "message": "Feature deleted" }
```

8. Tasks

Tasks now belong to a Feature (Epic), not directly to a Project.

POST	`/features/:feature_id/tasks`

Create a Task under a Feature.

Authentication: Bearer Token required

Request Body

```
{
  "title": "Implement Google OAuth",
  "description": "Add Sign in with Google flow",
  "status": "todo",
  "priority": "high",
  "due_date": "2026-05-01T00:00:00Z",
  "assignee_ids": [2, 3]
}
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 500,
    "title": "Implement Google OAuth",
    "feature_id": 100,
    "created_by": 1,
    "status": "todo",
    "priority": "high",
    "due_date": "2026-05-01T00:00:00Z",
    "assignees": [
      { "id": 2, "name": "ravi" },
      { "id": 3, "name": "kiran" }
    ],
    "created_at": "2026-04-18T13:00:00Z"
  }
}
```

GET	`/features/:feature_id/tasks`

List Tasks under a Feature.

Authentication: Bearer Token required

Parameters

Field	Type	Required	Description
status	string	No	todo / in_progress / done
priority	string	No	low / medium / high / critical
assignee_id	integer	No	Filter by assignee

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "tasks": [
      { "id": 500, "title": "Implement Google OAuth", "status": "todo", "priority":
"high" }
    ]
  }
}
```

GET	/tasks/:id

Get a Task by ID.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 500,
    "title": "Implement Google OAuth",
    "description": "Add Sign in with Google flow",
    "feature_id": 100,
    "status": "todo",
    "priority": "high",
    "due_date": "2026-05-01T00:00:00Z",
    "assignees": [{ "id": 2, "name": "ravi" }],
    "comments_count": 3
  }
}
```

PUT	/tasks/:id

Update a Task.

Authentication: Bearer Token required

Request Body

```
{
```

```
  "title": "Implement Google & Apple OAuth",
  "priority": "critical"
}
```

Response (200 OK)

```
{
  "status": "success",
  "data": { "id": 500, "title": "Implement Google & Apple OAuth" }
}
```

PATCH `/tasks/:id/status`

Update only the status.

Authentication: Bearer Token required

Request Body

```
{ "status": "in_progress" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": { "id": 500, "status": "in_progress" }
}
```

POST `/tasks/:id/assignees`

Assign users to a Task.

Authentication: Bearer Token required

Request Body

```
{ "user_ids": [4, 5] }
```

Response (200 OK)

```
{
  "status": "success",
  "data": { "task_id": 500, "assignees": [{ "id": 4 }, { "id": 5 }] }
}
```

DELETE	/tasks/:id/assignees/:user_id

Unassign a user from a Task.

Authentication: Bearer Token required

Response (200 OK)

```
{ "status": "success", "message": "Assignee removed" }
```

DELETE	/tasks/:id

Delete a Task.

Authentication: Bearer Token required

Response (200 OK)

```
{ "status": "success", "message": "Task deleted" }
```

9. Comments

POST	/tasks/:task_id/comments

Add a comment to a Task.

Authentication: Bearer Token required

Request Body

```
{ "content": "I'll start on this tomorrow." }
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 900,
    "task_id": 500,
    "user_id": 2,
    "content": "I'll start on this tomorrow.",
    "created_at": "2026-04-18T14:00:00Z"
  }
}
```

GET	/tasks/:task_id/comments

List comments on a Task.

Authentication: Bearer Token required

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "comments": [
      { "id": 900, "user": { "id": 2, "name": "ravi" }, "content": "I'll start on this
tomorrow." }
    ]
  }
}
```

DELETE	/comments/:id

Delete a comment (author or admin only).

Authentication: Bearer Token required

Response (200 OK)

```
{ "status": "success", "message": "Comment deleted" }
```

10. Messages (Workspace Chat)

POST	/workspaces/:workspace_id/messages

Send a message in a workspace.

Authentication: Bearer Token required

Request Body

```
{ "content": "Standup at 10am everyone!" }
```

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "id": 1500,
    "workspace_id": 10,
    "sender_id": 1,
    "content": "Standup at 10am everyone!",
    "created_at": "2026-04-18T15:00:00Z"
  }
}
```

GET	/workspaces/:workspace_id/messages

List messages.

Authentication: Bearer Token required

Parameters

Field	Type	Required	Description
before	datetime	No	Cursor — return messages before this time
limit	integer	No	Default 50, max 100

Response (200 OK)

```
{
  "status": "success",
  "data": {
    "messages": [
      { "id": 1500, "sender": { "id": 1, "name": "anil" }, "content": "Standup at 10am
everyone!" }
    ],
    "next_cursor": "2026-04-18T14:50:00Z"
  }
```

```
}
```

11. Error Codes

Code	Status	Description
400	Bad Request	Invalid input or missing required fields
401	Unauthorized	Missing or invalid JWT token
403	Forbidden	User does not have permission for this action
404	Not Found	Resource (workspace / project / feature / task) does not exist
409	Conflict	Resource already exists (e.g., duplicate email)
422	Unprocessable	Validation failed
500	Server Error	Unexpected server-side error